THE CLEAN CODE

Jose Luis Martin Blanco
Dorin Daniel Radulescu Toea

https://github.com/jmb0049-code/CleanCode.git

WHAT IS A CODE SMELL?

Code smells are symptoms in the source code of a program that possibly indicate a deeper design problems. They are patterns that are generally considered to be against the principles of good design and programming.

THE SMELLED CODE

This is the initial code. Some problems with this code are:

- Giant methods that do everything.
- Variables that do nothing.
- Lack of privacity (all methods are public).

```java
import java.util.ArrayList;
import java.util.List;

public class OrderProcessor {

    // 1. TEMPORARY FIELD (OO Abuser): Variable used only in one specific method, polluting the class scope.
    private double temporaryDiscountValue;

    // 2. DEAD CODE (Dispensable): Unused variable that increases complexity without adding value.
    private int unusedCounter = 0;

    // 3. LONG METHOD (Bloater): This method is doing too many things (calculating, printing, and logic).
    public void process(String item, int qty, double price, String type, String email) {

        // 4. DUPLICATE CODE (Dispensable): This calculation logic is repeated further down.
        double total = qty * price;
        double tax = total * 0.21;
        double finalPrice = total + tax;

        // 5. SWITCH STATEMENT (OO Abuser): Hardcoded logic that should be replaced with Polymorphism.
        switch (type) {
            case "ELECTRONICS":
                finalPrice -= 10;
                break;
            case "CLOTHES":
                finalPrice -= 5;
                break;
        }

        // 6. FEATURE ENVY (Coupler): Method is more interested in the 'System.out' class than its own.
        if (email.contains("@")) {
            System.out.println("Sending email to " + email);
            System.out.println("Total: " + finalPrice);
        }

        // 7. DUPLICATE CODE (Dispensable): Repeating exactly the same calculations from above.
        double total2 = qty * price;
        double tax2 = total2 * 0.21;
        System.out.println("Audit log: " + (total2 + tax2));
    }

    // 8. INAPPROPRIATE INTIMACY (Coupler): Accessing private-like data of another class directly.
    // 9. LARGE CLASS (Bloater): This class is handling too many different responsibilities.
    public void intimateAccess(User user) {
        user.address = "New Street";
        user.internalID = "999";
    }
}

// 10. SHOTGUN SURGERY (Change Preventer): A single change in tax rules requires modifications in multiple places.
class TaxCalculator {
    public double applyIVA(double amount) {
        return amount * 1.21;
    }
}

// Extra - DIVERGENT CHANGE (Change Preventer): Class changes for multiple unrelated reasons (address, IDs).
class User {
    public String address;
    public String internalID;
}
```

CLASSIFICATION OF SMELLS

BLOATERS

Giant method and bigger class.

DISPENSABLES

Dead and duped code.

COUPLERS

Lack of privacity.

OO ABUSERS

Giant switch and temporary fields.

REFACTORING PROCESS

```
9  -    // 1. TEMPORARY FIELD (OO Abuser): Variable
       used only in one specific method, polluting the
       class scope.
10 -    private double temporaryDiscountValue;
11 +
```

```
7  +    // 1. FIXED TEMPORARY FIELD: Removed
       temporaryDiscountValue.
8
```

1. Remove temporary field.

```
9  -    // 2. DEAD CODE (Dispensable): Unused
       variable that increases complexity without adding
       value.
10 -    private int unusedCounter = 0;
```

```
9  +    // 2. FIXED DEAD CODE: Removed unusedCounter
       because it was't doing anything.
```

2. Remove unused code.

REFACTORING PROCESS

```diff
-     // 3. LONG METHOD (Bloater): This method is doing too many things (calculating, printing, and logic).
-     public void process(String item, int qty, double price, String type, String email) {
-
-         // 4. DUPLICATE CODE (Dispensable): This calculation logic is repeated further down.
-         double total = qty * price;
-         double tax = total * 0.21;
-         double finalPrice = total + tax;
-
-         // 5. SWITCH STATEMENT (OO Abuser): Hardcoded logic that should be replaced with Polymorphism.
-         switch (type) {
-             case "ELECTRONICS":
-                 finalPrice += 10;
-                 break;
-             case "CLOTHES":
-                 finalPrice += 5;
-                 break;
-         }
-
-         // 6. FEATURE ENVY (Coupler): Method is more interested in the 'System.out' class than its own.
-         if (email.contains("@")) {
-             System.out.println("Sending email to " + email);
-             System.out.println("Total: " + finalPrice);
-         }
-
-         // 7. DUPLICATE CODE (Dispensable): Repeating exactly the same calculations from above.
-         double total2 = qty * price;
-         double tax2 = total2 * 0.21;
-         System.out.println("Audit log: " + (total2 + tax2));
     }
```

```diff
+     // 3. FIXED LONG METHOD: process is shorter and delegate tasks.
+     public void process(Order order, User user) {
+
+         double finalPrice = calculatePrice(order);
+         //6. FIXED FEATURE ENVY: Logic separated into a unique method.
+         notifyUser(user, finalPrice);
+         //7. FIXED DUPLICATE CODE: Reuse the logic for the audit.
+         logAudit(order);
     }

+     // 4. FIXED DUPLICATE CODE: Single method for the core calculation.
+     private double calculatePrice(Order order)
+     {
+         double total = order.getQty() * order.getPrice();
+         double totalWithTax = applyTax(total);
+
+         // 5. FIXED SWITCH STATEMENT: Logic moved to Order class.
+         return order.applyDiscount(totalWithTax);
+     }
+
+
+
```

Fixed smells n° 3, 4, 6 & 7.

We fixed the long method (3) to be shorter and to delegate tasks. (Extract method)

We arranged duplicate code (4) for the calculation method.

We extracted method applied to addres Feature envy (6) (Information Expert).

We removed duplicate code (7). It was reusing the audit logic.

REFACTORING PROCESS

```
43 -    // 8. INAPPROPRIATE INTIMACY (Coupler): Accessing private-like data of another class directly.
44 -    // 9. LARGE CLASS (Bloater): This class is handling too many different responsibilities.
45      public void intimateAccess(User user) {
46 -        user.address = "New Street";
47 -        user.internalID = "999";
```

```
43 +    // 8. FIXED INAPPROPRIATE INTIMACY & 9. LARGE CLASS:
44 +    // Instead of touching 'user.address', we call a method on User.
45      public void intimateAccess(User user) {
46 +        user.updateProfile("New Street", "999");
```

Fixed smells n° 8 & 9.

Both methods were arranged by hiding implementation details within the object that owns the data. (Encapsulation)

```
62 -    public String getEmail() { return email; }
```

```
62 +    public String getEmail() { return email; }
63 +
64 +    // Fix for Smells 8 & 9: Encapsulation
65 +    public void updateProfile(String address, String internalID) {
66 +        this.address = address;
67 +        this.internalID = internalID;
68 +    }
69 + }
```

REFACTORING PROCESS

```java
+ // Helper class to organize data (Fixes Large Class)
+ class Order {
+     private int qty;
+     private double price;
+     private String type;
+
+     public int getQty() { return qty; }
+     public double getPrice() { return price; }
+
+     // Fix for 5. SWITCH STATEMENT: The object decides its own discount.
+     public double applyDiscount(double total) {
+         if ("ELECTRONICS".equals(type)) return total - 10;
+         if ("CLOTHES".equals(type)) return total - 5;
+         return total;
+     }
+ }
```

Fixed smell n° 5.

We extracted class to fix the smell large class and long parameter list.

We added a conditional instead of a switch, so the object now decides its own discount in his own method rather than having an external class.

REFACTORING PROCESS

```
- // 10. SHOTGUN SURGERY (Change Preventer): A single change in tax rules requires modifications in multiple places.
- class TaxCalculator {
-     public double applyIVA(double amount) {
          return amount * 1.21;
      }
  }

- // Extra - DIVERGENT CHANGE (Change Preventer): Class changes for multiple unrelated reasons (address, IDs).
  class User {
-     public String address;
-     public String internalID;
- }
```

```
+     // 10. FIXED SHOTGUN SURGERY: Centralized tax calculation.
+     private double applyTax(double amount) {
          return amount * 1.21;
      }
  }

+ // Fixed DIVERGENT CHANGE: User now controls its own data.
  class User {
+     private String address;
+     private String internalID;
+     private String email;

+
+     public String getEmail() { return email; }
```

Fixed smell n° 10 & EXTRA.

Resolved Shotgun Surgery by centralizing the tax calculation. (10).

Fixed Divergent Change by encapsulating data within User class. (Extra).

REFACTORING BENEFITS

TESTING

The code is easier to test.

READABILITY

The code is easier to read.

FLEXIBILITY

The code is scalable.

END